

Loading the Vault

How to fill a file-based AI memory with the knowledge you already have

Last time I shared the structure — the three-layer folder that gives an AI a memory you own. This is the other half: how you get your actual knowledge into it. Scrubbed of anything specific to me, ready to copy. There's a companion file that hands this whole process to your AI to run; this guide is so you understand what it's doing.

The reframe that makes it tractable: **you don't have a knowledge problem, you have an extraction problem.** The knowledge already exists — scattered across months of AI chats, half-written docs, repo READMEs, decisions you made in some conversation and never recorded. You're not creating it. You're freeing it and putting it where your AI can use it.

The idea, in one paragraph

Filling the vault is two moves, and the entire discipline is in not collapsing them into one. First you **extract**: point the AI at one source and have it pull the structured signal into a staging file you review — not your wiki. Then you **compile**: turn the reviewed staging file into durable wiki pages, and have the AI propose the decisions worth recording rather than writing them silently. You stay the editor; the AI does the typing. The reason for the staging step is simple — an AI mining months of messy history will sometimes get something wrong, and the staging file is where you catch it before it reaches your permanent memory. Do the import once, then keep it fed: a minute per new source beats an afternoon of re-doing it.

The two steps

Step	Reads	Writes to	Output	The rule
1 - Extract	one source + the anchor	<code>02-synthesized/</code> <code><project>/</code> (staging)	a structured extraction you review	cite every source; mark inferences; don't touch the wiki
2 - Compile	the reviewed extraction + the anchor	<code>10-<domain>/</code> <code><project>/</code> (wiki) + <code>_meta/decisions/</code>	wiki pages + proposed decision records	the anchor wins on conflict; propose, never mint

The staging layer (`02-synthesized/`) is the quarantine. Raw history goes in, structured-but-unverified knowledge comes out, you check it, and only then does anything reach the wiki. Skip it and a single confident hallucination becomes permanent.

Step 1 — Extract

Pick **one** source at a time — an exported chat history, a project folder, a repo. If it's worth keeping verbatim, drop a copy into `01-sources/` first (date-stamped, never edited again). Then run the extraction.

The prompt:

```
Mine <source> for the project <project> into a staging file. Treat the anchor as
authoritative — where a source disagrees with it, the anchor wins and you flag the
conflict; never silently average them. Do NOT write to the wiki in this step.

Write to 02-synthesized/<project>/<YYYY-MM-DD>-extraction.md with this shape:
- sources: cite every chat (title + approx date), every repo/file you read
- Current state: phase / status / blockers, reconciled with the anchor
- Decisions made: D1..DN, each with WHAT / WHEN / WHY / ALTERNATIVES considered
- Still open: the unresolved questions (count them)
- Contradictions found: "source A says X (date); source B says Y; anchor resolves to Z"
- Private context — do NOT compile: anything that must never reach the wiki (or "none")

Cite provenance for everything. Mark anything you inferred as (inferred). Don't invent
state — if it isn't in a source or the anchor, it doesn't go in. When done, summarize in
chat (decisions, contradictions, open-question count) and wait for my review.
```

The output is a working document, not a finished one. Read it. Fix what the AI got wrong — that's the whole point of the step. The extraction is allowed to be long and a little messy; it's the raw ore, not the jewelry.

The extraction's shape, as a template:

```
---
type: extraction
updated: 2026-06-15
sources:
  - anchor: 02-synthesized/<project>/_anchor-2026-06-10.md
  - chats: "<title>, ~2026-05"
  - repo: "~/code/<project>/README.md"
tags: [<project>, extraction]
---

## Current state (reconciled with the anchor)
- Phase / status / blockers – two or three lines.

## Decisions that have been made
D1. WHAT: <decision> · WHEN: <date> · WHY: <rationale> · ALTERNATIVES: <considered &
rejected>

## Still open
- <question> (total: N)

## Contradictions found
- <source A says X (date); source B says Y (date). Anchor resolves to: Z.>

## Private context – do NOT compile
- <sensitive items that stay in staging> (or: "none")
```

Step 2 — Compile

Only after you've reviewed the extraction. The prompt:

```
Read the reviewed extraction and the anchor. Produce wiki pages under
10-<domain>/<project>/:
- frontmatter on every page (type: wiki, updated, tags, and sources: — which extraction
  it came from; missing sources is the most common gap)
- wikilink related pages; keep them specific (concrete details over vague summaries);
  split anything past ~500 lines
- carry the (inferred) marks through; the anchor wins on any conflict

Never create a decision record silently. At the end, output proposed ADRs:
  "PROPOSED ADR-NNN: <title> — <summary>. Sources: <extraction line>. Create? [y/n]"
Propose one only for a decision that is settled, involved a real choice with
alternatives,
and has documented rationale. Not for trivially-true facts; not for anything still open.

Then update index.md (one line on the project's state) and append a line to log.md.
Wait for my approval before writing any ADR.
```

You approve decisions explicitly — "create 1, 3, 5; skip 2, 4" — and only then do they become files. This is the load-bearing habit: **the AI proposes, you ratify**. A knowledge base that mints its own decisions is a knowledge base you stop trusting.

The rule that holds it together

When sources disagree — and across months of history, they will — your **current** statement of where things stand wins. An old chat says the launch is in May; your anchor says it slipped to July. July wins, and the AI surfaces the conflict instead of averaging two dates into a fiction.

Three habits enforce it: - **Provenance on everything**. Every extraction cites its sources; every wiki page cites the extraction it came from. You can always trace a claim back to where it came from. - **Inference is marked**. Anything the AI concluded rather than read says `(inferred)`. You never mistake the AI's guess for the record. - **The anchor is authoritative**. It's the short, current-state truth you maintain by hand. Everything else gets checked against it.

Keeping it fed

The import is one afternoon. The value is in keeping it current — and that's a minute, not an afternoon. Three loops:

A new source shows up — a chat worth keeping, a doc, a decision made in passing. Drop it in `01-sources/` if it's worth keeping. Extract just the delta (what's new), update the

anchor's current-state, bump its `updated:` date. Compile only if durable knowledge actually changed; propose an ADR for any genuinely new decision. The one habit that keeps the whole thing honest: at the end of any session where something changed, one line into the anchor — "we changed X today."

You start a new project — spin up `10-<domain>/<new-project>/` and a fresh anchor (or use the Builder for the skeleton). Then run Step 1 against whatever already exists for it.

Every so often (monthly is plenty) — a light review: which anchors have a stale `updated:` date? Any wiki pages nothing links to? Any drift between an extraction and the page it produced? Any page missing its `sources:` ? Fix or flag. This is what stops slow rot.

Where to point it first

Don't try to load everything at once. Pick the one project whose context you most wish your AI already had — the active one, the messy one. Load that. Feel the difference in the next session. Then do the next. The system earns its keep one project at a time, and a half-loaded vault that you actually use beats a complete one you built in a weekend and never fed again.

Credits & sources

The parts aren't mine; the assembly is.

- **Capture** → **distill** → **express** is the personal-knowledge-management loop (Tiago Forte's second brain, on the older progressive-summarization and Zettelkasten lineage). Extract-then-compile is that loop pointed at an AI.
- **Decision records** (the ADR format — a decision, its context, its alternatives) come from the software-architecture world (Michael Nygard's ADRs).
- **"Stage before you commit"** is just the discipline every data pipeline already uses — land raw, transform in a reviewable layer, publish only what's checked.

What's mine is fitting them together for AI memory, and the one rule that keeps it trustworthy: the AI proposes, you ratify. Steal the rest.

This kit is free — copy it, adapt it, share it. It's the working method, not a product.